

RAPPORT DE PROJET DE FIN D'ANNÉE

Filière : 4IADATA — 4ème année Ingénierie Data & Intelligence
Artificielle

Détection des Équipements de Protection Individuelle (EPI) par Intelligence Artificielle

Réalisé par :

Ettayeb BENCHEKROUNE • Taha EL BAL • Ilyass SEKOUNI

Encadré par :

M. El Mehdi ANIQ

Remerciements

La réalisation de ce projet de fin d'année n'aurait pas été possible sans le soutien et l'encadrement de plusieurs personnes à qui nous souhaitons exprimer notre gratitude.

Nous tenons tout d'abord à remercier chaleureusement notre encadrant, M. El Mehdi ANIQ, pour sa disponibilité constante, ses orientations éclairées et ses précieux conseils tout au long de ce projet. Sa rigueur académique et son expertise dans le domaine de l'intelligence artificielle ont été une source d'inspiration et de motivation permanente.

Nous remercions également l'ensemble du corps enseignant de l'EMSI Marrakech, et plus particulièrement les professeurs de la filière 4IADATA, dont les enseignements de qualité nous ont fourni les bases théoriques et pratiques nécessaires à l'accomplissement de ce travail.

Nos remerciements vont aussi à nos familles et à nos proches pour leur soutien moral indéfectible, leur patience et leur encouragement tout au long de cette année universitaire exigeante.

Enfin, nous remercions la communauté open source et les chercheurs qui ont rendu disponibles les outils et les datasets utilisés dans ce projet, notamment les contributeurs du framework Ultralytics YOLO et les annotateurs du dataset utilisé.

Résumé

Ce rapport présente le développement d'un système de détection automatique des Équipements de Protection Individuelle (EPI) à base d'intelligence artificielle, dans le cadre d'une application de surveillance de la sécurité sur les chantiers et dans les environnements industriels.

Le projet repose sur l'utilisation du modèle de détection d'objets YOLOv8 (You Only Look Once), fine-tuné sur un dataset annoté comprenant trois classes : casque de protection (helmet), tête sans casque (head) et gilet de sécurité (vest). Le système est capable d'analyser des images ou des flux vidéo en temps quasi-réel, et de déterminer automatiquement si une personne respecte les normes de sécurité en vigueur.

Les résultats obtenus sont encourageants, avec un mAP@0.5 global de 0.846 pour l'ensemble des classes. Le système est en mesure de classer chaque individu détecté comme "SAFE" (port du casque et du gilet confirmé) ou "NOT SAFE" (absence d'un ou plusieurs équipements), offrant ainsi une aide précieuse aux responsables de sécurité sur site.

Ce travail s'inscrit dans une démarche de prévention des accidents du travail et de modernisation des processus de contrôle de conformité, à travers l'intégration de solutions d'intelligence artificielle dans des contextes industriels réels.

Mots-clés : *Détection d'objets, YOLOv8, EPI, Sécurité industrielle, Computer Vision, Deep Learning.*

Abstract

This report presents the development of an AI-based automatic detection system for Personal Protective Equipment (PPE), designed for safety monitoring applications in construction sites and industrial environments.

The project is based on the YOLOv8 (You Only Look Once) object detection model, fine-tuned on an annotated dataset comprising three classes: safety helmet (helmet), unprotected head (head), and safety vest (vest). The system is capable of analyzing images or video streams in near real-time, and automatically determining whether a person complies with applicable safety regulations.

The results obtained are promising, with an overall mAP@0.5 of 0.846 across all classes. The system can classify each detected individual as "SAFE" (helmet and vest confirmed) or "NOT SAFE" (one or more missing equipment), providing valuable assistance to on-site safety managers.

This work contributes to workplace accident prevention and the modernization of compliance monitoring processes, through the integration of artificial intelligence solutions in real industrial contexts.

Keywords: *Object detection, YOLOv8, PPE, Industrial safety, Computer Vision, Deep Learning.*

Table des Matières

Remerciements	2
Résumé	3
Table des Matières	5
Liste des figures	8
Liste des tables	8
Introduction Générale	9
I. Contexte Général	10
1.1 Introduction	10
1.2 Contexte	10
1.3 Problématique	10
1.4 Objectifs	11
1.5 Méthodologie	11
1.6 Conclusion	11
II. État de l'Art	13
2.1 Introduction	13
2.2 Computer Vision	13
2.3 Deep Learning et Réseaux de Neurones Convolutionnels	13
2.4 Détection d'Objets	13
2.5 YOLO — You Only Look Once	14
2.5.1 Principe général	14
2.5.2 Évolution des versions	14
2.5.3 YOLOv8 — Architecture et innovations	15
2.6 Travaux existants sur la détection EPI	15
2.7 Conclusion	15
III. Présentation du Projet	16
3.1 Introduction	16
3.2 Description générale	16
3.3 Architecture du système	16
3.4 Technologies utilisées	16
3.5 Conclusion	17
IV. Dataset	18
4.1 Introduction	18
4.2 Description du dataset	18
4.3 Description des classes	18
4.4 Distribution des données	18
4.5 Exemples d'images du dataset	19
4.6 Conclusion	21
V. Prétraitement des Données	22

5.1 Introduction.....	22
5.2 Redimensionnement et normalisation	22
5.3 Augmentation des données.....	22
5.4 Format des annotations.....	22
5.5 Conclusion.....	23
VI. Modèle et Méthodologie	24
6.1 Introduction.....	24
6.2 Choix du modèle	24
6.3 Paramètres d'entraînement.....	24
6.4 Environnement d'entraînement	25
6.5 Stratégie d'évaluation	25
6.6 Conclusion.....	25
VII. Résultats et Évaluation	26
7.1 Introduction.....	26
7.2 Métriques globales	26
7.3 Courbes d'entraînement	26
7.4 Courbe Précision-Rappel	27
7.5 Courbes F1, Précision et Rappel en fonction du seuil de confiance	27
7.6 Matrice de Confusion	29
7.7 Tests visuels.....	30
7.8 Conclusion.....	32
VIII. Analyse du Système	33
8.1 Introduction.....	33
8.2 Points forts	33
8.3 Limites identifiées	33
8.4 Conclusion.....	34
IX. Application Pratique : SAFE / NOT SAFE	35
9.1 Introduction.....	35
9.2 Logique de décision	35
9.3 Visualisation des résultats.....	35
9.4 Cas d'usage réels.....	35
9.5 Conclusion.....	36
X. Améliorations Possibles	37
10.1 Introduction.....	37
10.2 Amélioration du dataset.....	37
10.3 Amélioration du modèle.....	37
10.4 Amélioration du système	37
10.5 Conclusion.....	38
Conclusion Générale	39

Bibliographie	40
Références Web	41
Annexes	42
Annexe A — Extrait du fichier de configuration d'entraînement (data.yaml)	42
Annexe B — Extrait du script d'entraînement Python	42
Annexe C — Extrait du script d'inférence et classification SAFE / NOT SAFE	42

Liste des figures

Figure 1- Distribution des instances par classe	19
Figure 2- Travailleurs portant casques et gilets avec un gilet manquant	20
Figure 3- Travailleur portant casque et gilet	20
Figure 4- Absence de gilet de sécurité	21
Figure 5- Absence de casque et de gilet	21
Figure 6- Métriques	27
Figure 7- Courbe BoxPr	27
Figure 8- Courbe F1-Confidence	28
Figure 9- Courbe Précision-Confidence	28
Figure 10- Courbe Rappel-Confidence	29
Figure 11- Matrice de confusion brute	29
Figure 12- Matrice de confusion normalisée	30
Figure 13- Travailleur avec casque et gilet	31
Figure 14- Travailleurs sans casque, avec gilet.....	31
Figure 15- Travailleur sans gilet.....	32
Figure 16- Extrait du fichier de configuration d'entraînement (data.yaml)	42
Figure 17- Extrait du script d'entraînement Python	42
Figure 18- Extrait du script d'inférence et classification SAFE / NOT SAFE.....	42

Liste des tables

Tableau 1- Les paramètres de l'entrainement	24
Tableau 2- Performances du modèle par classe	26

Introduction Générale

Dans un contexte industriel en constante évolution, la sécurité des travailleurs représente un enjeu majeur pour les entreprises. Malgré l'existence de réglementations strictes, le non-respect du port des Équipements de Protection Individuelle (EPI), tels que les casques et les gilets de sécurité, demeure une cause fréquente d'accidents de travail, parfois graves voire mortels.

Les méthodes traditionnelles de surveillance reposent principalement sur l'intervention humaine, ce qui présente plusieurs limites : manque de continuité, subjectivité, fatigue et incapacité à surveiller simultanément de larges zones. Face à ces contraintes, l'intégration de solutions basées sur l'intelligence artificielle, et plus particulièrement la vision par ordinateur (Computer Vision), constitue une alternative prometteuse.

Ce projet s'inscrit dans cette perspective et vise à concevoir un système automatisé capable de détecter en temps réel la présence ou l'absence des EPI à partir d'images ou de flux vidéo. Pour ce faire, nous nous appuyons sur le modèle YOLOv8, reconnu pour ses performances élevées en détection d'objets en temps réel.

L'objectif principal est de développer un système intelligent permettant non seulement de détecter les équipements de sécurité (casque, gilet), mais également de classifier les individus en deux catégories : SAFE (conforme) et NOT SAFE (non conforme), afin d'assister les responsables de sécurité dans la prévention des risques.

Ce rapport présente l'ensemble des étapes de réalisation du projet, depuis l'étude de l'état de l'art jusqu'à l'analyse des résultats obtenus, en passant par la conception du système, le traitement des données et l'entraînement du modèle.

I. Contexte Général

1.1 Introduction

Dans un environnement industriel en constante évolution, la sécurité des travailleurs constitue un enjeu majeur pour les entreprises et les organismes de régulation. Malgré l'existence de normes strictes imposant le port des Équipements de Protection Individuelle (EPI), leur respect reste difficile à garantir sur le terrain, en raison de la complexité des environnements de travail et des limites des méthodes de contrôle traditionnelles.

Les avancées récentes en intelligence artificielle, et plus particulièrement en vision par ordinateur, offrent aujourd'hui de nouvelles perspectives pour automatiser la surveillance de la sécurité. Ces technologies permettent d'analyser des images ou des flux vidéo en temps réel afin de détecter des situations à risque.

Dans ce chapitre, nous présentons le contexte général du projet, en abordant les enjeux liés à la sécurité industrielle, la problématique étudiée ainsi que les objectifs poursuivis. Cette analyse permet de mieux comprendre la motivation et la pertinence du système développé.

1.2 Contexte

Dans un monde industriel en constante évolution, la sécurité des travailleurs demeure une priorité absolue pour les entreprises, les organismes de réglementation et les responsables de chantiers. Les accidents du travail liés à l'absence ou au mauvais port des Équipements de Protection Individuelle (EPI) représentent encore aujourd'hui une source majeure de blessures graves, voire mortelles. Casques, gilets réfléchissants, harnais de sécurité — autant d'équipements dont le port obligatoire reste difficile à contrôler efficacement avec les seuls moyens humains, surtout dans des environnements vastes et dynamiques.

Face à cette réalité, l'intelligence artificielle, et en particulier la Computer Vision, offre des perspectives prometteuses pour automatiser la surveillance de la conformité aux règles de sécurité. La capacité des modèles de deep learning à analyser des images ou des vidéos en temps réel ouvre la voie à des systèmes de surveillance intelligents, capables de détecter instantanément les infractions et d'alerter les responsables concernés.

1.3 Problématique

Les contrôles manuels de sécurité sont coûteux, subjectifs, et ne peuvent pas couvrir en permanence l'ensemble d'un site industriel. Un agent de sécurité humain ne peut surveiller simultanément qu'un nombre limité de zones, laissant des angles morts potentiellement dangereux. De plus, la fatigue et les biais humains peuvent conduire à des omissions.

La question centrale de ce projet est donc : Comment concevoir un système automatisé, robuste et précis, capable de détecter en temps réel la présence ou l'absence d'équipements de protection individuelle sur des personnes observées par des caméras, afin d'assister les responsables de sécurité dans leur mission de contrôle ?

1.4 Objectifs

Les objectifs de ce projet peuvent être résumés comme suit :

- ☐ Développer un modèle de détection d'objets capable d'identifier les casques de sécurité (helmet), les têtes non protégées (head) et les gilets de sécurité (vest) dans des images.
- ☐ Entraîner et évaluer ce modèle sur un dataset représentatif des conditions réelles de chantier.
- ☐ Implémenter une logique de décision permettant de classifier automatiquement chaque individu détecté comme "SAFE" ou "NOT SAFE".
- ☐ Analyser les performances du modèle à travers des métriques standards de la détection d'objets (précision, rappel, mAP).
- ☐ Identifier les limites du système et proposer des pistes d'amélioration.

1.5 Méthodologie

Notre approche méthodologique suit le cycle classique d'un projet de machine learning appliqué à la Computer Vision :

- ☐ Collecte et analyse d'un dataset annoté adapté à la problématique.
- ☐ Prétraitement des données et augmentation pour améliorer la robustesse du modèle.
- ☐ Sélection et fine-tuning d'un modèle YOLOv8 pré-entraîné.
- ☐ Entraînement sur les données du projet et suivi des métriques.
- ☐ Évaluation des performances sur un jeu de données de test indépendant.
- ☐ Intégration d'une logique de classification SAFE / NOT SAFE et tests visuels.

1.6 Conclusion

Ce chapitre a permis de mettre en évidence les enjeux majeurs liés à la sécurité des travailleurs dans les environnements industriels, ainsi que les limites des approches traditionnelles de surveillance. Face à ces défis, l'intégration de solutions basées sur l'intelligence artificielle apparaît comme une réponse pertinente et innovante.

La problématique formulée souligne la nécessité de concevoir un système automatisé capable de détecter efficacement le port des équipements de protection individuelle en temps réel. Les

objectifs définis orientent ainsi le développement d'une solution basée sur la vision par ordinateur, visant à améliorer la prévention des risques.

Dans la suite de ce rapport, nous allons explorer les concepts théoriques et les travaux existants qui constituent la base de notre approche, avant de détailler la conception et l'implémentation du système proposé.

II. État de l'Art

2.1 Introduction

Ce chapitre présente les concepts fondamentaux et les travaux existants liés à la détection des équipements de protection individuelle par intelligence artificielle. Il permet de situer notre projet dans le contexte des avancées récentes en vision par ordinateur et en deep learning, ainsi que d'identifier les méthodes les plus adaptées à notre problématique.

2.2 Computer Vision

La Computer Vision, ou vision par ordinateur, est une branche de l'intelligence artificielle qui cherche à doter les machines de la capacité à interpréter et comprendre le contenu visuel du monde réel — images, vidéos, flux de caméras [8]. Elle regroupe un ensemble de techniques permettant d'extraire des informations sémantiques à partir de données visuelles brutes.

Les applications de la Computer Vision sont aujourd'hui omniprésentes : reconnaissance faciale, véhicules autonomes, diagnostic médical par imagerie, contrôle qualité industriel, surveillance de sécurité, et bien d'autres. L'essor de la Computer Vision est intimement lié aux avancées du deep learning, qui a profondément transformé la manière d'aborder les problèmes de perception visuelle.

2.3 Deep Learning et Réseaux de Neurones Convolutionnels

Le deep learning repose sur des architectures de réseaux de neurones artificiels à plusieurs couches, capables d'apprendre automatiquement des représentations hiérarchiques à partir de grandes quantités de données [8]. Là où les approches classiques de vision par ordinateur nécessitaient une ingénierie manuelle des features (extraction de caractéristiques), les réseaux de neurones profonds apprennent ces représentations de manière autonome.

Les Réseaux de Neurones Convolutionnels (CNN, Convolutional Neural Networks) constituent la pierre angulaire de la Computer Vision moderne. Leur architecture, inspirée du cortex visuel biologique, est particulièrement bien adaptée au traitement d'images. Les couches de convolution extraient progressivement des features de plus en plus abstraites — des bords et textures dans les premières couches, à des formes et objets complexes dans les couches profondes. Des architectures comme VGG, ResNet, EfficientNet et MobileNet ont marqué les grandes étapes de l'évolution des CNN [4][5].

2.4 Détection d'Objets

La détection d'objets (object detection) est une tâche qui consiste à localiser et à classifier simultanément un ou plusieurs objets dans une image. Contrairement à la simple classification d'image qui produit une étiquette globale, la détection d'objets fournit pour chaque objet identifié une boîte englobante (bounding box) précisant sa position et ses dimensions, ainsi qu'une étiquette de classe associée à un score de confiance [7].

L'évolution de la détection d'objets a suivi deux grandes familles d'approches :

- ❑ Les méthodes à deux stages (two-stage detectors) comme R-CNN et Faster R-CNN, qui proposent d'abord des régions candidates avant de les classifier. Ces méthodes sont généralement précises mais plus lentes.
- ❑ Les méthodes à un seul stage (one-stage detectors), comme SSD (Single Shot Detector) ou YOLO (You Only Look Once), qui effectuent la détection et la localisation en une seule passe sur l'image. Elles sont plus rapides et mieux adaptées aux applications temps réel.

2.5 YOLO — You Only Look Once

2.5.1 Principe général

YOLO (You Only Look Once) est une famille d'algorithmes de détection d'objets introduite en 2016 par Joseph Redmon et ses collaborateurs [1]. Son nom illustre parfaitement son fonctionnement : l'image est analysée en une seule passe par le réseau, ce qui lui confère une vitesse d'inférence remarquable, adaptée au traitement vidéo en temps réel.

Le principe fondamental de YOLO consiste à diviser l'image en une grille de cellules. Chaque cellule est responsable de la prédiction de boîtes englobantes et de probabilités de classes pour les objets dont le centre se trouve dans cette cellule. En une seule inférence, le réseau produit l'ensemble des détections pour l'image entière.

2.5.2 Évolution des versions

Depuis sa création, YOLO a connu de nombreuses itérations, chacune apportant des améliorations significatives en termes de précision et de vitesse [2] :

- ❑ YOLOv1 à v4 : Les premières versions ont posé les bases de l'architecture et ont progressivement amélioré la précision sur les objets petits et multiples.
- ❑ YOLOv5 (Ultralytics, 2020) : Réimplémentation en PyTorch, très accessible, avec un excellent compromis vitesse/précision.
- ❑ YOLOv7 (2022) : Amélioration de l'architecture avec des techniques d'augmentation avancées.

- YOLOv8 (Ultralytics, 2023) : Nouvelle génération, avec une architecture repensée, support natif des tâches de classification, détection, segmentation et pose estimation, et une API Python intuitive [3].

2.5.3 YOLOv8 — Architecture et innovations

YOLOv8 est la version utilisée dans notre projet. Elle introduit plusieurs innovations architecturales majeures par rapport à ses prédécesseurs :

- Backbone CSPDarknet amélioré : Extraction de features plus efficace grâce aux modules C2f (Cross Stage Partial with 2 feature flows).
- Neck PAN-FPN : Combinaison d'un Path Aggregation Network (PAN) et d'un Feature Pyramid Network (FPN) pour une meilleure détection multi-échelle.
- Tête de détection Anchor-Free : YOLOv8 abandonne les ancres prédéfinies au profit d'une approche sans ancres, ce qui simplifie l'entraînement et améliore la généralisation.
- Perte DFL (Distribution Focal Loss) : Une fonction de perte avancée pour une régression plus précise des boîtes englobantes.

YOLOv8 est disponible en plusieurs tailles (nano, small, medium, large, xlarge), permettant d'adapter le compromis vitesse/précision aux contraintes matérielles de déploiement.

2.6 Travaux existants sur la détection EPI

La détection automatique des EPI par Computer Vision est un domaine de recherche actif. Plusieurs travaux ont exploré l'utilisation de YOLO et d'autres détecteurs pour la surveillance de la sécurité sur les chantiers [9][10]. Des datasets publics comme le Safety Helmet Wearing Dataset (SHWD) ou le CHV dataset ont été largement utilisés dans la littérature. Les approches les plus récentes atteignent des performances mAP@0.5 comprises entre 0.80 et 0.95 selon les conditions d'acquisition et les classes considérées, confirmant la maturité de ces technologies pour des applications industrielles réelles.

2.7 Conclusion

À travers cette étude de l'état de l'art, nous avons identifié que les méthodes basées sur le deep learning, et en particulier les modèles de détection d'objets comme YOLO, offrent un excellent compromis entre précision et rapidité. Ces caractéristiques les rendent particulièrement adaptées aux applications en temps réel telles que la surveillance de sécurité. Ces observations ont guidé le choix du modèle YOLOv8 dans le cadre de notre projet.

III. Présentation du Projet

3.1 Introduction

Ce chapitre a pour objectif de présenter de manière globale le projet développé, en décrivant son fonctionnement, son architecture ainsi que les technologies utilisées. Il permet de comprendre comment les différentes composantes du système interagissent pour répondre à la problématique de détection des EPI.

3.2 Description générale

Notre projet consiste en le développement d'un système complet de détection automatique des EPI. L'objectif opérationnel est simple : à partir d'une image ou d'un flux vidéo, le système doit être capable d'identifier les personnes présentes dans la scène, de détecter la présence ou l'absence des équipements de sécurité requis, et de produire une alerte visuelle en conséquence.

Le choix d'une architecture YOLO s'est imposé naturellement pour sa capacité à effectuer des détections en temps quasi-réel, une caractéristique essentielle pour une application de surveillance continue. Le fine-tuning d'un modèle pré-entraîné sur nos données spécifiques nous a permis de bénéficier du transfer learning, réduisant ainsi considérablement le volume de données d'entraînement nécessaire et le temps de calcul.

3.3 Architecture du système

Le pipeline de notre système se décompose en plusieurs étapes clés :

- ❑ Acquisition : Entrée d'une image ou d'une vidéo provenant d'une caméra de surveillance ou d'un fichier.
- ❑ Inférence : Passage de l'image à travers le modèle YOLOv8 fine-tuné, qui produit les boîtes englobantes, les classes et les scores de confiance pour chaque détection.
- ❑ Post-traitement : Application d'un seuil de confiance et de la suppression des non-maximum (NMS) pour filtrer les détections redondantes.
- ❑ Décision SAFE / NOT SAFE : Logique applicative qui agrège les détections par zone corporelle pour déterminer si chaque personne respecte les règles de sécurité.
- ❑ Visualisation : Affichage des résultats avec les boîtes englobantes colorées (vert pour SAFE, rouge pour NOT SAFE) et les étiquettes correspondantes.

3.4 Technologies utilisées

Le projet a été développé en Python, en s'appuyant sur les bibliothèques suivantes :

- ❑ Ultralytics YOLOv8 : Framework principal pour l'entraînement et l'inférence du modèle de détection.
- ❑ PyTorch : Backend de deep learning utilisé par Ultralytics.
- ❑ OpenCV : Bibliothèque de Computer Vision pour le traitement d'images et la visualisation des résultats.
- ❑ NumPy / Pandas : Manipulation des données et des métriques.
- ❑ Matplotlib / Seaborn : Génération des graphiques de performance.

3.5 Conclusion

Cette présentation du projet met en évidence une architecture claire et modulaire, permettant de traiter efficacement les images et de produire des résultats exploitables en temps réel. Le choix des technologies, notamment YOLOv8 et OpenCV, assure à la fois performance et flexibilité dans le développement du système.

IV. Dataset

4.1 Introduction

La qualité d'un modèle de détection repose en grande partie sur la qualité et la représentativité des données utilisées pour son entraînement. Ce chapitre décrit le dataset exploité dans ce projet, en détaillant ses caractéristiques, ses classes ainsi que la distribution des données.

4.2 Description du dataset

Le dataset utilisé pour l'entraînement et l'évaluation du modèle est un ensemble d'images annotées représentant des environnements industriels et de chantier variés. Les images couvrent des conditions d'éclairage diverses, des angles de vue multiples et des densités de personnes variées, afin d'assurer une bonne généralisation du modèle.

Chaque image est accompagnée de son fichier d'annotations au format YOLO (fichiers .txt contenant les coordonnées normalisées des boîtes englobantes et les identifiants de classe correspondants). Le dataset est divisé en trois sous-ensembles : entraînement (train), validation (val) et test (test), selon une répartition classique [14].

4.3 Description des classes

Le modèle est entraîné à reconnaître trois classes distinctes :

- ☐ **helmet** : Désigne un casque de sécurité porté sur la tête d'une personne. La détection de cette classe indique le respect de l'obligation de port du casque.
- ☐ **head** : Désigne la tête d'une personne non couverte par un casque. La présence de cette classe sans casque associé constitue une infraction aux règles de sécurité.
- ☐ **vest** : Désigne un gilet de sécurité (gilet réfléchissant ou de chantier) porté par une personne. La détection de cette classe valide le port de la protection corporelle.

La logique de classification SAFE / NOT SAFE repose sur la combinaison de ces trois classes : une personne est considérée comme SAFE si et seulement si au moins un casque (helmet) et un gilet (vest) sont détectés en association avec elle. Dans tous les autres cas, elle est classifiée NOT SAFE.

4.4 Distribution des données

Le tableau ci-dessous résume la distribution des instances par classe dans le jeu de données d'évaluation :

Class	Images	Instances	Box(P	R	mAP50	mAP50-95) :
all	1964	18500	0.869	0.865	0.846	0.519
helmet	1358	4803	0.915	0.863	0.861	0.568
vest	358	761	0.798	0.864	0.827	0.555
head	626	12936	0.892	0.867	0.851	0.434

Figure 1- Distribution des instances par classe

Le dataset contient environ 1964 images annotées avec un total de 18500 instances.

On observe une forte disparité dans la distribution des classes : la classe head est de loin la plus représentée avec 12 936 instances, suivie de helmet avec 4 803 instances, et enfin vest avec seulement 761 instances. Cette distribution reflète la réalité du terrain où de nombreuses personnes apparaissent dans les images sans nécessairement porter tous leurs équipements.

4.5 Exemples d'images du dataset

Afin d'illustrer la nature des données utilisées pour l'entraînement du modèle, nous présentons ci-dessous quelques exemples d'images issues du dataset, accompagnées de leurs annotations.

Ces images montrent la diversité des situations rencontrées, notamment en termes de conditions d'éclairage, de position des travailleurs et de présence ou absence des équipements de protection individuelle.

Les annotations sont représentées sous forme de boîtes englobantes (bounding boxes), associées à chaque classe (helmet, head, vest). Elles constituent l'information essentielle permettant au modèle d'apprendre à localiser et classer les objets d'intérêt.

Ces exemples mettent en évidence la complexité du problème, notamment dans les cas de chevauchement d'objets, de faible visibilité ou de diversité des environnements, ce qui justifie l'utilisation de techniques avancées de deep learning.



Figure 2- Travailleurs portant casques et gilets avec un gilet manquant



Figure 3- Travailleur portant casque et gilet



Figure 4- Absence de gilet de sécurité



Figure 5- Absence de casque et de gilet

4.6 Conclusion

L'analyse du dataset met en évidence un déséquilibre significatif entre les classes, ce qui peut influencer les performances du modèle. Toutefois, la diversité des images permet d'assurer une certaine robustesse face aux conditions réelles, constituant ainsi une base solide pour l'entraînement du modèle.

V. Prétraitement des Données

5.1 Introduction

Avant l'entraînement du modèle, il est nécessaire de préparer les données afin de garantir leur compatibilité avec l'architecture choisie et d'améliorer la capacité de généralisation du modèle. Ce chapitre présente les différentes étapes de prétraitement appliquées aux données.

5.2 Redimensionnement et normalisation

Avant l'entraînement, toutes les images sont redimensionnées à une résolution fixe de 640×640 pixels, qui constitue la taille d'entrée standard du modèle YOLOv8. Les valeurs des pixels sont normalisées entre 0 et 1. Cette normalisation est effectuée automatiquement par le pipeline d'entraînement d'Ultralytics.

5.3 Augmentation des données

Afin d'améliorer la robustesse du modèle face aux variations du monde réel et de limiter le risque de surapprentissage (overfitting), un ensemble de techniques d'augmentation de données a été appliqué lors de l'entraînement :

- ❑ Retournement horizontal aléatoire (horizontal flip) : Simule des positions symétriques des travailleurs dans la scène.
- ❑ Variations d'exposition et de saturation (HSV augmentation) : Améliore la robustesse aux variations d'éclairage et aux conditions météorologiques.
- ❑ Mosaic augmentation : Technique propre à YOLO qui combine quatre images en une seule, favorisant la détection d'objets petits et augmentant la diversité contextuelle.
- ❑ Random crop et translation : Améliore la détection des objets en bordure d'image.
- ❑ Perspective transform : Simule différents angles de prise de vue des caméras de surveillance.

Ces augmentations sont appliquées en ligne (on-the-fly) pendant l'entraînement, ce qui signifie que chaque batch d'images est transformé de manière différente à chaque époque, multipliant ainsi efficacement la diversité des données sans augmenter le volume de stockage.

5.4 Format des annotations

Les annotations sont fournies au format YOLO, avec un fichier texte par image contenant une ligne par objet détecté. Chaque ligne contient l'identifiant de classe suivi des coordonnées normalisées de la boîte englobante (centre x, centre y, largeur, hauteur), toutes comprises

entre 0 et 1 par rapport aux dimensions de l'image. Ce format est directement compatible avec le pipeline d'entraînement d'Ultralytics YOLOv8.

5.5 Conclusion

Les techniques de prétraitement et d'augmentation des données mises en œuvre ont permis d'enrichir le dataset et de renforcer la robustesse du modèle face aux variations du monde réel. Ces étapes constituent un élément clé dans l'amélioration des performances globales du système.

VI. Modèle et Méthodologie

6.1 Introduction

Ce chapitre décrit les choix méthodologiques adoptés pour le développement du modèle, notamment la sélection de l'architecture YOLOv8, les paramètres d'entraînement ainsi que l'environnement utilisé. Il détaille également la stratégie d'évaluation mise en place pour mesurer les performances du modèle.

6.2 Choix du modèle

Parmi les différentes variantes de YOLOv8 disponibles (nano, small, medium, large, xlarge), notre choix s'est porté sur YOLOv8m (medium). Cette variante offre un excellent équilibre entre la précision de détection et la vitesse d'inférence, ce qui la rend particulièrement adaptée à notre cas d'usage où les performances temps réel sont importantes sans pour autant disposer d'un matériel de calcul extrêmement puissant.

Le modèle a été initialisé avec des poids pré-entraînés sur le dataset COCO (Common Objects in Context) [6], qui contient 80 classes d'objets du quotidien. Cette initialisation par transfer learning permet au modèle de partir d'une représentation riche du monde visuel, et d'adapter ensuite ses poids à notre tâche spécifique de détection d'EPI.

6.3 Paramètres d'entraînement

L'entraînement a été réalisé avec les paramètres suivants :

Tableau 1- Les paramètres de l'entraînement

Paramètre	Valeur
Modèle de base	YOLOv8m (pré-entraîné COCO)
Nombre d'époques	30
Taille d'image	640 × 640 pixels
Batch size	16
Optimizer	SGD (Stochastic Gradient Descent)
Learning rate initial	0.01
Learning rate final	$0.01 \times \text{lrf}$
Momentum	0.937
Weight decay	0.0005
Augmentation mosaic	Activée
Seuil de confiance (inférence)	0.314 (optimal F1)

Paramètre	Valeur
IoU threshold (NMS)	0.45

6.4 Environnement d'entraînement

L'entraînement a été réalisé sur un environnement GPU, ce qui a permis de réduire significativement le temps de calcul par rapport à un entraînement sur CPU. Le framework Ultralytics YOLOv8 gère automatiquement l'utilisation du GPU via PyTorch (CUDA). Le suivi des métriques a été réalisé en temps réel tout au long des 30 époques d'entraînement.

6.5 Stratégie d'évaluation

L'évaluation du modèle est réalisée sur le jeu de données de test, complètement indépendant des données d'entraînement et de validation. Les métriques utilisées sont les métriques standard de la détection d'objets :

- Précision (Precision, P) : Proportion de détections positives qui sont correctes. Mesure la fiabilité des détections.
- Rappel (Recall, R) : Proportion d'objets réels qui ont été correctement détectés. Mesure la complétude de la détection.
- F1-Score : Moyenne harmonique de la précision et du rappel, offrant un indicateur de performance global.
- mAP@0.5 (mean Average Precision) : Aire sous la courbe précision-rappel pour un seuil IoU de 0.5. Métrique de référence en détection d'objets.
- mAP@0.5-0.95 : Moyenne du mAP pour des seuils IoU allant de 0.5 à 0.95 par pas de 0.05. Métrique plus stricte reflétant la qualité de la localisation.

6.6 Conclusion

Les choix méthodologiques adoptés, en particulier l'utilisation de YOLOv8m et du transfer learning, ont permis d'obtenir un modèle performant tout en optimisant le temps d'entraînement. La stratégie d'évaluation mise en place garantit une analyse fiable et rigoureuse des résultats obtenus.

VII. Résultats et Évaluation

7.1 Introduction

Ce chapitre présente les résultats obtenus après l'entraînement du modèle, ainsi que leur analyse à travers différentes métriques et visualisations. Il vise à évaluer la performance du système et à identifier ses points forts et ses limites.

7.2 Métriques globales

Après 30 époques d'entraînement, les performances du modèle évaluées sur le jeu de test sont les suivantes :

Tableau 2- Performances du modèle par classe

Classe	Images	Instances	Précision	Rappel	mAP@0.5	mAP@0.5:0.95
all	1964	18500	0.869	0.865	0.846	0.519
helmet	1358	4803	0.915	0.863	0.861	0.568
vest	358	761	0.798	0.864	0.827	0.555
head	626	12936	0.892	0.867	0.851	0.434

Ces résultats montrent des performances solides du modèle pour l'ensemble des classes. La classe helmet obtient les meilleures performances avec une précision de 0.915 et un mAP@0.5 de 0.861, ce qui s'explique par la forme distinctive et relativement uniforme des casques de sécurité. La classe vest, bien que moins représentée dans le dataset, affiche des résultats corrects avec un mAP@0.5 de 0.827. La classe head, malgré un grand nombre d'instances, présente un mAP@0.5:0.95 plus faible (0.434), reflétant la difficulté intrinsèque à localiser précisément les têtes dans des scènes complexes.

7.3 Courbes d'entraînement

L'évolution des métriques au cours de l'entraînement est visible sur les courbes suivantes. Les courbes de loss (box_loss, cls_loss, dfl_loss) montrent une convergence régulière et cohérente, tant sur les données d'entraînement que sur les données de validation, sans signe notable de surapprentissage.

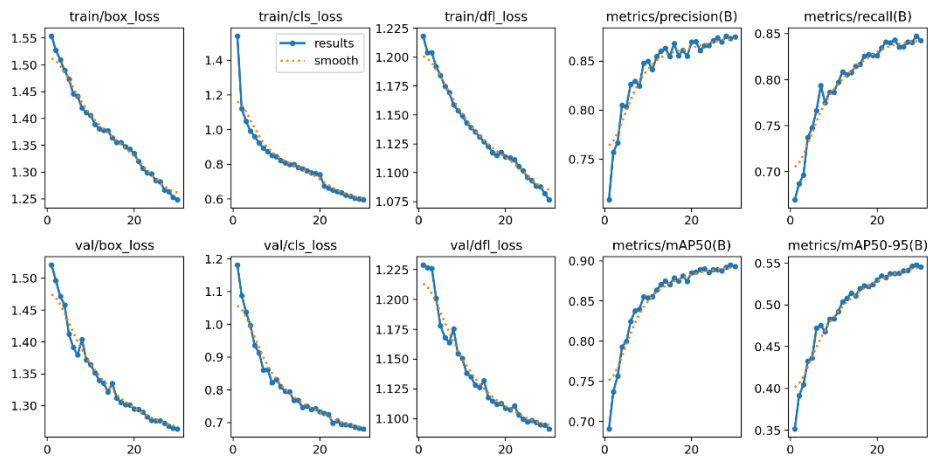


Figure 6- Métriques

Ces résultats sont comparables aux travaux récents utilisant YOLO pour la détection des EPI, confirmant la pertinence de l'approche adoptée.

7.4 Courbe Précision-Rappel

La courbe Précision-Rappel illustre le compromis entre précision et rappel pour différents seuils de confiance. Une courbe s'approchant du coin supérieur droit (précision = 1, rappel = 1) indique un modèle idéal. Les courbes obtenues montrent de bonnes performances pour les trois classes, avec des aires sous la courbe élevées.

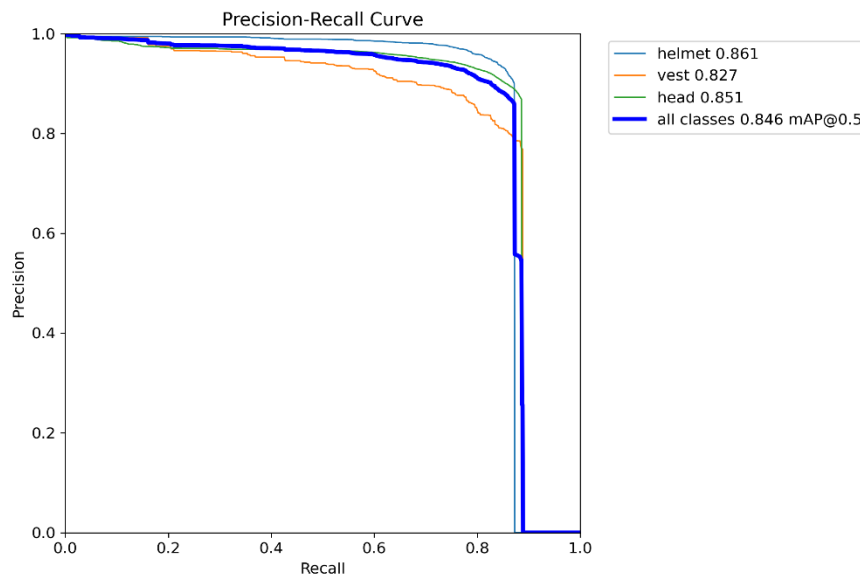


Figure 7- Courbe BoxPr

7.5 Courbes F1, Précision et Rappel en fonction du seuil de confiance

La courbe F1-Confidence permet d'identifier le seuil de confiance optimal minimisant simultanément les faux positifs et les faux négatifs. Pour notre modèle, le seuil optimal pour l'ensemble des classes est de 0.314, donnant un F1-Score de 0.87.

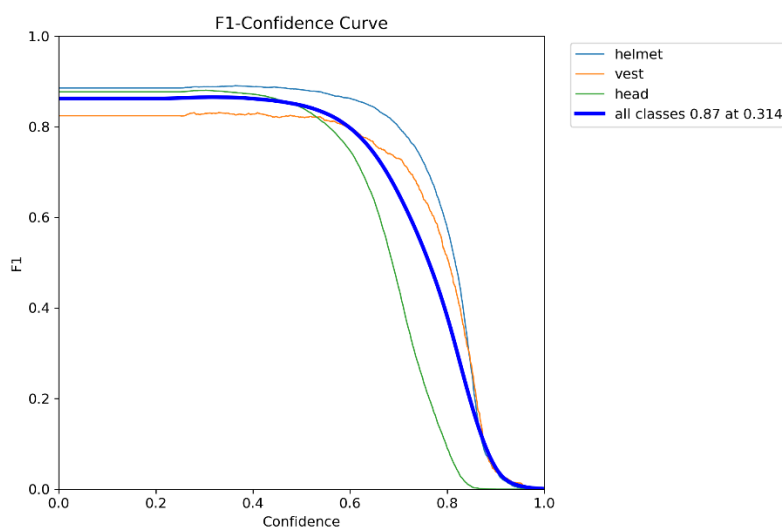


Figure 8- Courbe F1-Confidence

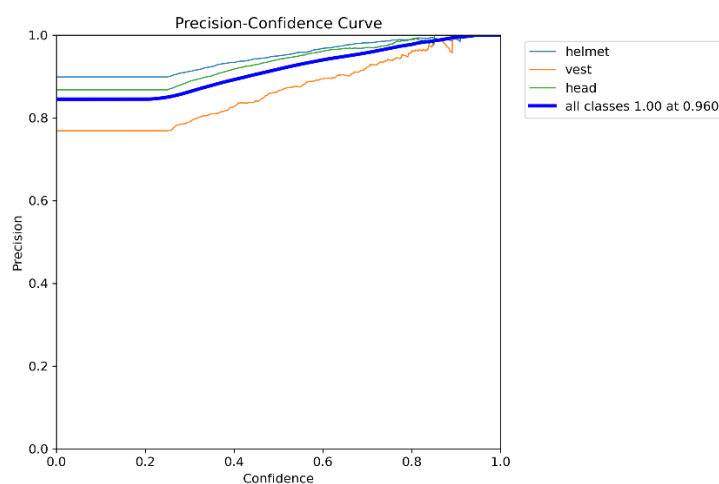


Figure 9- Courbe Précision-Confidence

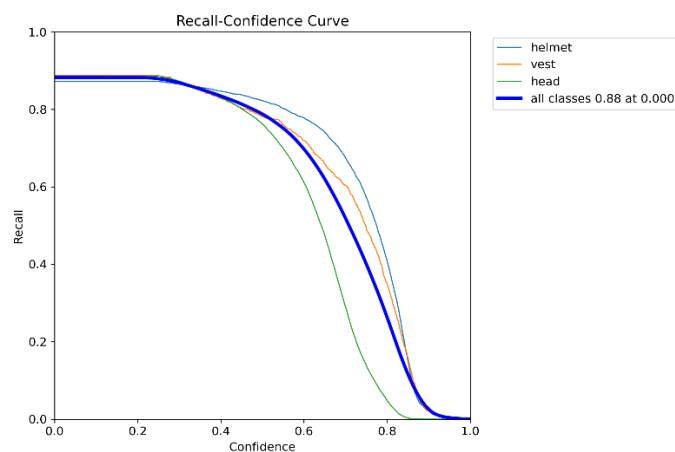


Figure 10- Courbe Rappel-Confidence

7.6 Matrice de Confusion

La matrice de confusion fournit une vue détaillée des performances par classe, en mettant en évidence les confusions les plus fréquentes entre classes. La version normalisée permet une interprétation plus aisée des taux d'erreur.

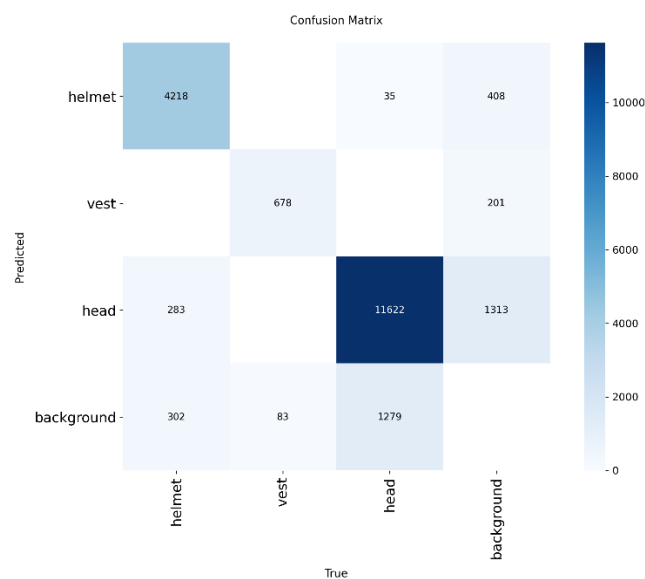


Figure 11- Matrice de confusion brute



Figure 12- Matrice de confusion normalisée

L'analyse de la matrice de confusion normalisée révèle :

- Helmet : 88% de vraies détections correctes. 6% sont confondues avec la classe head, et 6% sont perdues en background. Ce résultat s'explique par la possible confusion entre une tête partiellement visible et un casque.
- Vest : 89% de détections correctes. 11% sont attribuées au background, ce qui s'explique par la diversité des couleurs et des types de gilets présents dans le dataset.
- Head : 90% de détections correctes, mais avec 68% des backgrounds prédits comme head — indiquant que le modèle a tendance à sur-détecter les têtes dans certaines zones de background. Ce phénomène est commun pour des classes naturelles sans frontière nette avec le fond.

7.7 Tests visuels

Les tests visuels sur des images réelles confirment la capacité du système à détecter correctement les équipements et à produire la classification SAFE / NOT SAFE attendue.

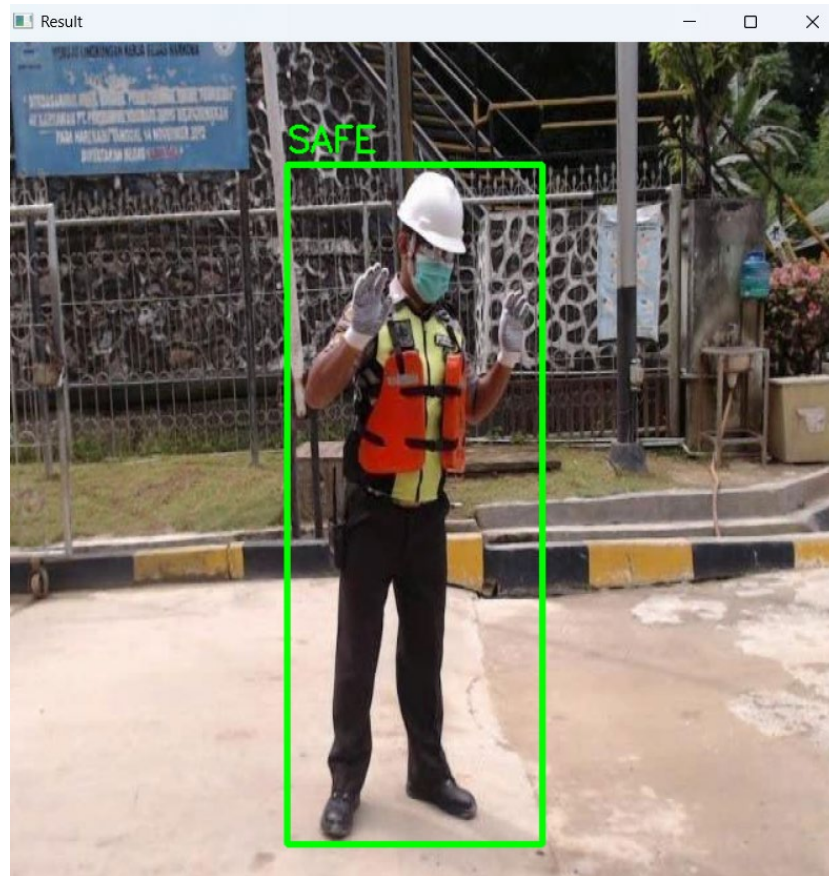


Figure 13- Travailleur avec casque et gilet

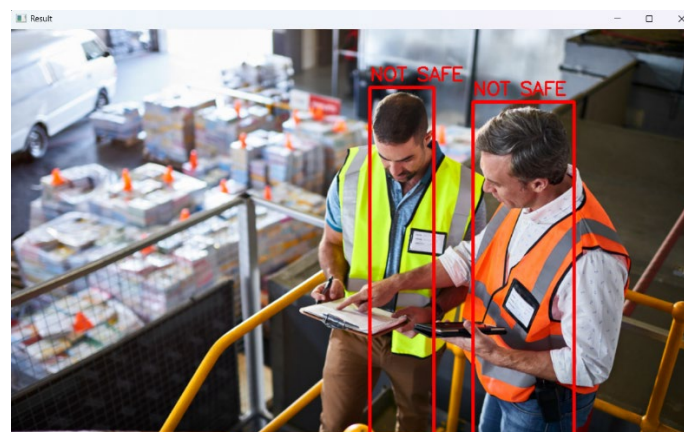


Figure 14- Travailleurs sans casque, avec gilet

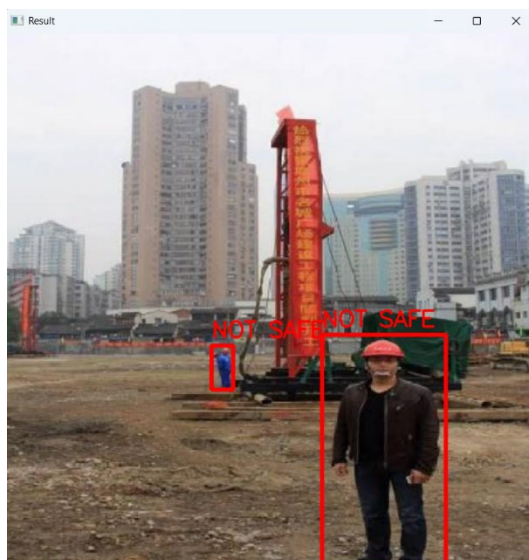


Figure 15- Travailleur sans gilet

7.8 Conclusion

Les résultats obtenus démontrent l'efficacité du modèle pour la détection des EPI, avec des performances globales satisfaisantes. Malgré certaines limites, notamment liées au déséquilibre des classes, le système présente un potentiel réel pour des applications en conditions réelles.

VIII. Analyse du Système

8.1 Introduction

Ce chapitre propose une analyse critique du système développé, en mettant en évidence ses points forts ainsi que ses limites. Il permet de mieux comprendre les performances observées et d'identifier les axes d'amélioration.

8.2 Points forts

L'analyse des résultats met en lumière plusieurs atouts significatifs du système développé :

- Performances globales solides : Un $mAP@0.5$ de 0.846 toutes classes confondues est un résultat compétitif, en ligne avec les travaux académiques récents sur des problématiques similaires.
- Convergence stable de l'entraînement : Les courbes de loss montrent une progression régulière sans oscillations importantes, témoignant d'une bonne configuration des hyperparamètres.
- Bonne précision sur la classe critique : La classe helmet, qui est la plus critique pour la sécurité, obtient les meilleures performances avec une précision de 91.5%, limitant les fausses alertes.
- Vitesse d'inférence : L'architecture YOLOv8m permet une inférence rapide, compatible avec un traitement vidéo en temps quasi-réel sur un matériel standard équipé d'un GPU.
- Système end-to-end : De l'acquisition de l'image à l'affichage du verdict SAFE / NOT SAFE, le pipeline est complet et opérationnel.

8.3 Limites identifiées

Malgré des résultats encourageants, plusieurs limites méritent d'être soulignées :

- Déséquilibre des classes : La forte disproportion entre le nombre d'instances de head (12 936) et de vest (761) a pu influencer défavorablement les performances sur la classe vest et introduire des biais dans le processus d'apprentissage.
- Faux positifs sur le background (classe head) : La matrice de confusion révèle que 68% des prédictions de fond (background) sont incorrectement classifiées comme head, ce qui peut engendrer des fausses alertes dans des scènes sans personnes.
- Sensibilité aux conditions d'acquisition : Le modèle peut rencontrer des difficultés dans des conditions d'éclairage extrêmes, avec des occlusions importantes ou pour des personnes très éloignées de la caméra.

- ❑ Nombre d'époques limité : Avec seulement 30 époques d'entraînement, il est possible que le modèle n'ait pas atteint sa performance maximale. Un entraînement plus long pourrait encore améliorer les métriques.
- ❑ Absence de tracking temporel : Le système traite chaque frame de manière indépendante. L'intégration d'un algorithme de tracking (comme DeepSORT) permettrait de suivre les individus dans le temps et d'améliorer la cohérence des détections.

8.4 Conclusion

L'analyse du système montre que celui-ci est globalement performant et adapté à la problématique étudiée. Toutefois, certaines limitations techniques subsistent, ouvrant la voie à des améliorations futures pour renforcer sa précision et sa robustesse.

IX. Application Pratique : SAFE / NOT SAFE

9.1 Introduction

Ce chapitre présente l'application pratique du modèle à travers la mise en place d'une logique de décision permettant de classer les individus en fonction du respect des règles de sécurité. Il illustre l'utilisation concrète du système développé.

9.2 Logique de décision

L'application pratique du modèle repose sur une logique de décision simple mais efficace, implémentée en post-traitement des détections YOLO. Pour chaque frame analysée, le système procède comme suit :

- ☐ Toutes les détections sont récupérées avec leurs boîtes englobantes, classes et scores de confiance (filtrés au-dessus du seuil de 0.314).
- ☐ Pour chaque détection de la classe head (tête sans casque), le système vérifie si une boîte englobante de classe helmet est présente dans la même zone spatiale ou à proximité immédiate.
- ☐ La présence d'une boîte de type vest dans la région corporelle de la personne est également vérifiée.
- ☐ Une personne est classifiée SAFE si elle porte à la fois un helmet et un vest. Sinon, elle est classifiée NOT SAFE.

9.3 Visualisation des résultats

Les résultats de la classification sont affichés visuellement sur l'image ou la vidéo traitée :

- ☐ Une boîte englobante VERTE est tracée autour de la personne avec le label "SAFE", indiquant le respect des règles de sécurité.
- ☐ Une boîte englobante ROUGE est tracée autour de la personne avec le label "NOT SAFE", signalant une non-conformité.

Cette visualisation intuitive permet aux responsables de sécurité de saisir immédiatement la situation sur le chantier, sans nécessiter d'expertise technique particulière.

9.4 Cas d'usage réels

En pratique, ce système peut être intégré à différents types de déploiements :

- ☐ Surveillance par caméras IP sur les chantiers de construction, avec analyse en temps réel des flux vidéo.
- ☐ Contrôle d'accès aux zones à risque, avec déclenchement automatique d'une alarme ou d'un refus d'accès si les EPI ne sont pas détectés.

- Audit de sécurité sur des enregistrements vidéo, pour analyser après coup la conformité des travailleurs sur une période donnée.
- Intégration dans des tableaux de bord de gestion de la sécurité, fournissant des statistiques sur les taux de conformité par zone ou par équipe.

9.5 Conclusion

La mise en œuvre de la logique SAFE / NOT SAFE permet de transformer les résultats du modèle en informations directement exploitables. Cette approche rend le système particulièrement utile dans un contexte industriel réel, en facilitant la prise de décision rapide.

X. Améliorations Possibles

Fort des enseignements tirés de ce projet, plusieurs pistes d'amélioration ont été identifiées pour des versions futures du système :

10.1 Introduction

Ce chapitre explore les différentes pistes d'amélioration du système, tant au niveau des données, du modèle que de son déploiement. Il s'inscrit dans une logique d'évolution continue du projet.

10.2 Amélioration du dataset

- ❑ Rééquilibrage des classes : Augmenter le nombre d'images annotées pour la classe vest afin de réduire le déséquilibre de représentation et d'améliorer les performances sur cette classe.
- ❑ Diversification des scènes : Intégrer davantage d'images dans des conditions d'éclairage difficiles (nuit, contre-jour, intérieur sombre) et avec des occlusions variées.
- ❑ Extension des classes : Ajouter d'autres EPI comme les gants de protection, les lunettes de sécurité, ou les harnais anti-chute.

10.3 Amélioration du modèle

- ❑ Augmentation du nombre d'époques : Un entraînement sur 50 à 100 époques, avec early stopping, pourrait encore améliorer les métriques.
- ❑ Utilisation de YOLOv8l ou YOLOv8x : Les variantes plus larges du modèle offrent généralement une meilleure précision, au prix d'un temps d'inférence légèrement supérieur.
- ❑ Techniques de data augmentation avancées : Mixup, CutMix ou la génération de données synthétiques par GAN pourraient enrichir le dataset.

10.4 Amélioration du système

- ❑ Intégration d'un tracker : L'ajout d'un algorithme de tracking multi-objets (ByteTrack, DeepSORT) permettrait de suivre les individus entre les frames et d'améliorer la cohérence des décisions SAFE / NOT SAFE dans le temps.
- ❑ Déploiement sur edge devices : Optimisation du modèle (quantification, pruning, export ONNX / TensorRT) pour un déploiement sur des caméras intelligentes ou des systèmes embarqués (Jetson Nano, Raspberry Pi).

- ❑ Interface de gestion : Développement d'une interface web ou mobile permettant aux responsables de sécurité de visualiser les alertes en temps réel, de consulter l'historique des détections et de générer des rapports automatisés.
- ❑ Alertes en temps réel : Intégration d'un système de notifications (email, SMS, push) pour alerter automatiquement le responsable de sécurité dès qu'une infraction est détectée.

10.5 Conclusion

Les perspectives d'amélioration identifiées montrent qu'il est possible d'augmenter significativement les performances du système. Ces évolutions pourraient permettre un déploiement à grande échelle dans des environnements industriels variés.

Conclusion Générale

Ce projet de fin d'année nous a permis de concevoir et de développer un système complet de détection automatique des Équipements de Protection Individuelle, en s'appuyant sur les techniques les plus récentes de Computer Vision et de Deep Learning. En fine-tunant le modèle YOLOv8m sur un dataset annoté spécifique à notre problématique, nous avons réussi à construire un système capable de détecter les casques de sécurité, les têtes non protégées et les gilets de sécurité avec des performances solides ($mAP@0.5 = 0.846$).

Au-delà de la performance technique, ce projet démontre la viabilité et la pertinence de l'intelligence artificielle pour répondre à des enjeux concrets de sécurité industrielle. La classification automatique SAFE / NOT SAFE offre une solution pratique et opérationnelle pour assister les responsables de sécurité dans leur mission de surveillance et de prévention des accidents du travail.

Ce travail nous a également permis d'approfondir nos compétences en data science appliquée, en ingénierie des modèles de deep learning, en gestion de pipeline de données et en évaluation rigoureuse des systèmes d'apprentissage automatique — des compétences directement transférables dans le cadre professionnel que nous allons rejoindre.

Les perspectives d'amélioration identifiées — rééquilibrage du dataset, extension des classes EPI, intégration d'un tracker, déploiement sur edge devices — ouvrent la voie à un système encore plus robuste et polyvalent, capable de s'adapter à des environnements industriels variés et exigeants.

En définitive, ce projet illustre comment l'intelligence artificielle peut constituer un levier puissant pour améliorer la sécurité au travail, réduire les risques d'accidents et moderniser les processus de contrôle de conformité dans le secteur industriel.

Bibliographie

- [1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- [2] Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv preprint arXiv:2004.10934.
- [3] Ultralytics. (2023). *YOLOv8 Documentation*. Disponible sur : <https://docs.ultralytics.com>
- [4] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- [5] Simonyan, K., & Zisserman, A. (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. International Conference on Learning Representations (ICLR).
- [6] Lin, T.-Y., Maire, M., Belongie, S., et al. (2014). *Microsoft COCO: Common Objects in Context*. European Conference on Computer Vision (ECCV).
- [7] Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J., & Zisserman, A. (2010). *The Pascal Visual Object Classes (VOC) Challenge*. International Journal of Computer Vision.
- [8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [9] Zhang, S., Wu, X., & Luo, X. (2019). *Real-time safety helmet detection based on deep learning*. IEEE Access.
- [10] Nath, N. D., Behzadan, A. H., & Paal, S. G. (2020). *Deep learning for site safety: Real-time detection of personal protective equipment*. Automation in Construction.

Références Web

[11] Ultralytics YOLOv8 GitHub Repository : <https://github.com/ultralytics/ultralytics>

[12] PyTorch Documentation : <https://pytorch.org/docs>

[13] OpenCV Documentation : <https://docs.opencv.org>

[14] Kaggle Dataset (Hardhat & Vest Detection Dataset):
<https://www.kaggle.com/datasets/muhammetzahitaydn/hardhat-vest-dataset-v3>

Annexes

Annexe A — Extrait du fichier de configuration d'entraînement (data.yaml)

```
! data.yaml
1  path: /kaggle/working/balanced_dataset
2  train: train/images
3  val: val/images
4  test: test/images
5
6  nc: 3
7  names: ['Helmet', 'Vest', 'Head']
```

Figure 16- Extrait du fichier de configuration d'entraînement (data.yaml)

Annexe B — Extrait du script d'entraînement Python

```
results = model.train(
    data="/kaggle/working/data.yaml",
    epochs=30,
    imgsz=640,
    batch=16,
    device=0
)
```

Figure 17- Extrait du script d'entraînement Python

Annexe C — Extrait du script d'inférence et classification SAFE / NOT SAFE

```
# ===== AFFICHAGE =====
for person_id, info in new_person_info.items():
    x1, y1, x2, y2 = map(int, info['box'])

    if info['display_status'] == 'SAFE':
        label = "SAFE"
        color = (0, 255, 0)
    else:
        label = "NOT SAFE"
        color = (0, 0, 255)

    # Afficher aussi le compteur de frames pour debug
    cv2.rectangle(frame, (x1, y1), (x2, y2), color, 3)
    cv2.putText(frame, label, (x1, y1 - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.85, color, 2)

    # Petit texte pour voir le filtrage en action (debug)
    cv2.putText(frame, f"conf: {info['consecutive_frames']}/{FRAMES_REQUIRED}",
                (x1, y2 + 15), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (200, 200, 200), 1)
```

Figure 18- Extrait du script d'inférence et classification SAFE / NOT SAFE